

JavaScript pour la boucle

Dans ce chapitre, vous apprendrez à répéter une série d'actions à l'aide de boucles en JavaScript.

Différents types de boucles en JavaScript

Les boucles sont utilisées pour exécuter le même bloc de code encore et encore, tant qu'une certaine condition est remplie. L'idée de base derrière une boucle est d'automatiser les tâches répétitives dans un programme pour économiser du temps et des efforts. JavaScript prend désormais en charge cinq types de boucles différents :

- **while** - boucle dans un bloc de code tant que la condition spécifiée est vraie.
- **do...while** — parcourt un bloc de code une fois ; puis la condition est évaluée. Si la condition est vraie, l'instruction est répétée tant que la condition spécifiée est vraie.
- **for** — parcourt un bloc de code jusqu'à ce que le compteur atteigne un nombre spécifié.
- **for...in** — parcourt les propriétés d'un objet.
- **for...of** — boucle sur des objets itérables tels que des tableaux, des chaînes, etc.

Dans les sections suivantes, nous discuterons en détail de chacune de ces instructions de boucle.

La boucle while

Il s'agit de l'instruction de bouclage la plus simple fournie par JavaScript.

La boucle **while** parcourt un bloc de code tant que la condition spécifiée est vraie. Dès que la condition échoue, la boucle est arrêtée. La syntaxe générique de la boucle **while** est :

```
while ( condition ) {  
    // Code à exécuter  
}
```

L'exemple suivant définit une boucle qui continuera à s'exécuter tant que la variable **i** est inférieure ou égale à 5. La variable **i** augmentera de **1** à chaque exécution de la boucle :

```
let i = 1;  
while(i <= 5) {  
    document.write("<p>Le nombre est: " + i + "</p>");  
    i++;  
}
```

Remarque : Assurez-vous que la condition spécifiée dans votre boucle finit par devenir fausse. Sinon, la boucle n'arrêtera jamais d'itérer, ce qui est connu sous le nom de **boucle infinie**. Une erreur courante est d'oublier d'incrémenter la variable compteur (variable **i** dans notre cas).

La boucle do...while

La boucle **do-while** est une variante de la boucle **do-while**, qui évalue la condition à la fin de chaque itération de la boucle. Avec une boucle **do-while**, le bloc de code est exécuté une fois, puis la condition est évaluée. Si la condition est vraie, l'instruction est répétée tant que la condition spécifiée évaluée est vraie. La syntaxe générique de la boucle **do-while** est :

```
do {  
    // Code à exécuter  
}  
while ( condition );
```

Le code JavaScript dans l'exemple suivant définit une boucle qui commence par **i=1**. Il imprimera ensuite la sortie et augmentera la valeur de la variable **i** de **1**. Après cela, la condition est évaluée et la boucle continuera à s'exécuter tant que la variable **i** est inférieure ou égale à **5**.

```
do {  
    document.write("<p>Le nombre est " + i + "</p>");  
    i++;  
}  
while(i <= 5);
```

Différence entre while et do...while

La boucle **while** diffère de la boucle **do-while** d'une manière importante - avec une boucle **while**, la condition à évaluer est testée au début de chaque itération de boucle, donc si l'expression conditionnelle est évaluée à faux, la boucle ne sera jamais exécutée.

Avec une boucle **do-while**, en revanche, la boucle sera toujours exécutée une fois même si l'expression conditionnelle est évaluée à faux, car contrairement à la boucle **while**, la condition est évaluée à la fin de l'itération de la boucle plutôt qu'au début.

La boucle for

La boucle **for** répète un bloc de code tant qu'une certaine condition est remplie. Il est généralement utilisé pour exécuter un bloc de code un certain nombre de fois. Sa syntaxe est :

```
for ( initialisation ; condition ; incrémentation ) {  
    // Code à exécuter  
}
```

Les paramètres de l'instruction de boucle **for** ont les significations suivantes :

- **initialisation** — elle est utilisée pour initialiser les variables du compteur, et évaluée une fois sans condition avant la première exécution du corps de la boucle.
- **condition** — elle est évaluée au début de chaque itération. S'il est évalué à **true**, les instructions de boucle s'exécutent. S'il est évalué à **false**, l'exécution de la boucle se termine.
- **incrément** — il met à jour le compteur de boucle avec une nouvelle valeur chaque fois que la boucle s'exécute.

L'exemple suivant définit une boucle qui commence par $i=1$. La boucle continuera jusqu'à ce que la valeur de la variable i soit inférieure ou égale à 5. La variable i augmentera de 1 à chaque exécution de la boucle

```
for(let i=1; i<=5; i++) {  
    document.write("<p>Le nombre est: " + i + "</p>");  
}
```

La boucle for est particulièrement utile pour itérer sur un tableau. L'exemple suivant vous montre comment imprimer chaque élément du tableau JavaScript.

```
// Un tableau avec quelques éléments  
let fruits = ["Pomme", "Banana", "Mangue", "Orange", "Papaye"];  
  
// Boucle sur tous les éléments du tableau  
for(let i=0; i<fruits.length; i++) {  
    document.write("<p>" + fruits[i] + "</p>");  
}
```

La boucle for..in

La boucle **for-in** est un type spécial de boucle qui itère sur les propriétés d'un objet ou les éléments d'un tableau. La syntaxe générique de la boucle **for-in** est:

```
for(variable in object) {  
    // Code à exécuté  
}
```

Le compteur de boucle, c'est-à-dire la variable dans la boucle **for-in**, est une chaîne, pas un nombre. Il contient le nom de la propriété courante ou l'index de l'élément courant du tableau.

L'exemple suivant vous montrera comment parcourir toutes les propriétés d'un objet JavaScript.

```
// Un objet avec quelques propriétés  
let person = {"prénom": "John", "nom": "Doe", "age": "36"};  
  
// Boucle sur toutes les propriétés de l'objet  
for(let i in person) {  
    document.write("<p>" + i + " = " + person[i] + "</p>");  
}
```


De même, vous pouvez parcourir les éléments d'un tableau, comme ceci

```
// Un tableau avec quelques éléments
let fruits = ["Pomme", "Banana", "Mangue", "Orange", "Papaye"];

// Parcourez en boucle tous les éléments du tableau
for(let i in fruits) {
    document.write("<p>" + fruits[i] + "</p>");
}
```

Remarque : La boucle `for-in` ne doit pas être utilisée pour itérer sur un tableau où l'ordre de l'index est important. Il vaut mieux utiliser une boucle `for` avec un index numérique.

La boucle for...of

ES6 introduit une nouvelle boucle **for-of** qui nous permet d'itérer très facilement sur des tableaux ou d'autres objets itérables (par exemple des chaînes). De plus, le code à l'intérieur de la boucle est exécuté pour chaque élément de l'objet itérable.

L'exemple suivant vous montrera comment parcourir des tableaux et des chaînes à l'aide de cette boucle.

```
// Iterating over array
let letters = ["a", "b", "c", "d", "e", "f"];

for(let letter of letters) {
    document.write(letter + "<br>");
}
document.write("<hr>");

// Iterating over string
let greet = "Hello World!";

for(let character of greet) {
    document.write(character + "<br>");
}
```

Remarque : La boucle `for...of` ne fonctionne pas avec les objets car ils ne sont pas itérables. Si vous souhaitez parcourir les propriétés d'un objet, vous pouvez utiliser la boucle `for-in`.